UNIVERSIDAD AUTONOMA

DE NUEVO LEÓN

FACULTAD DE INGENIERÍA

MECÁNICA Y ELÉCTRICA

Inteligencia Artificial y Redes Neuronales

Actividad #5

Procesamiento de conjunto de datos

Carrera	Matricula	Nombre	Hora
IMC	2103795	Diego Nedved Saucedo de la Torre	N5
IMC	2095410	Jesus Valdez Hurtado	N5
IMC	2098337	Rosio Nicole Gonzalez Guerrero	V6
IMC	2177946	Ximena Guedea Saldaña	V6
IMC	2120827	Vanessa Mendoza Ramirez	V6

Semestre Enero – Junio 2026

Cd. Universitaria, San Nicolás de los Garza a 6 de marzo del 2026

Descripción

El primer paso consistió en configurar el entorno de trabajo en **Google Colab** importando las librerías esenciales de Python: **Pandas** para la manipulación de tablas, **NumPy** para operaciones numéricas y **Matplotlib** para la generación de gráficos.

1. Configuración del entorno

En este paso se importaron las librerías pandas, numpy y matplotlib, que son fundamentales para el análisis de datos en Python.

```
[1]
✓ 0 s
# IMC 2098337 Rosio Nicole Gonzalez Guerrero
# IMC 2095410 Jesus Valdez Hurtado
# IMC 2120827 Vanessa Mendoza Ramírez
# IMC 2103795 Diego Nedved Saucedo de la Torre
# IMC 2177946 Ximena Guedea Saldaña
```

+ Código + Texto

```
[2]
✓ 0 s
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

url = "https://raw.githubusercontent.com/dilp90/InteligenciaArtificial_y_RedesNeuronales_UANL_FIME/9df5a5c93be47777b05ffc5cdaffcb2262d10013/MachineLearning/Datasets/robot_sensors.csv
# Cargar datos desde un archivo CSV
df = pd.read_csv(url)
```

2. Carga del dataset

Posteriormente se cargó el archivo robot_sensors.csv directamente desde GitHub y se mostraron las primeras filas de la tabla para verificar que los datos se hayan leído correctamente. Con esto se pudo observar la estructura general del dataset, confirmando que las columnas corresponden a tiempo, distancia, velocidad y temperatura.

```
[3]
✓ 0 s
print(df.head())

...
   Tiempo (s)  Distancia (cm)  Velocidad (cm/s)  Temperatura (°C)
0           0       43.708611          9.256646         23.925585
1           1       95.564288         27.095047         23.704682
2           2       75.879455         43.647292         33.593819
3           3       63.879264         36.611244         23.743193
4           4       24.041678         40.328057         24.079246
```

3. Limpieza de los datos

Esta fase se enfocó en garantizar la calidad de la información mediante dos acciones:

- **Gestión de omisiones:** Se descartaron todas las filas que presentaban campos vacíos o nulos para evitar errores en los cálculos.
- **Corrección de anomalías:** Se identificaron valores que no tenían lógica física (como velocidades o distancias negativas). En lugar de borrar esos registros, se optó por reemplazarlos con el **valor promedio** de su respectiva columna para mantener la integridad del dataset.

```
[4]
0 s
# Eliminar filas que contienen datos faltantes
df = df.dropna().copy()

# Verificar que no hay valores nulos restantes
print("valores nulos totales despues de eliminar filas con datos faltantes:",df.isnull().sum().sum())
```

```
valores nulos totales despues de eliminar filas con datos faltantes: 0
```

```
[7]
0 s
# Valores negativos en 'Velocidad (cm/s)' y reemplazados con la media
mean_velocity = df['Velocidad (cm/s)'].mean()
df.loc[df['Velocidad (cm/s)'] < 0, 'Velocidad (cm/s)'] = mean_velocity

# Valores negativos en 'Distancia (cm)' y reemplazados con la media
mean_distance = df['Distancia (cm)'].mean()
df.loc[df['Distancia (cm)'] < 0, 'Distancia (cm)'] = mean_distance

print("DataFrame después de filtrar valores atípicos (velocidad y distancia negativas):")
print(df.head())
```

```
DataFrame después de filtrar valores atípicos (velocidad y distancia negativas):
   Tiempo (s)  Distancia (cm)  Velocidad (cm/s)  Temperatura (°C)
0           0       43.708611          9.256646         23.925585
1           1       95.564288         27.095047         23.704682
2           2       75.879455         43.647292         33.593819
3           3       63.879264         36.611244         23.743193
4           4       24.041678         40.328057         24.079246
```

Después de esta limpieza, revisamos nuevamente el dataset para confirmar que todos los valores fueran válidos.

4. Normalización de los datos

Para que todas las mediciones fueran comparables, se aplicó una normalización que transformó los valores numéricos a un rango estricto entre **0 y 1**.

- El valor **0** representa el mínimo detectado y el **1** el máximo.
- Este proceso es vital en **Machine Learning** para evitar que una variable con números grandes opaque a las demás durante el análisis.

```
[8]
0 s
# Seleccionar las columnas numéricas (excluyendo 'Tiempo (s)')
numeric_cols = ['Distancia (cm)', 'Velocidad (cm/s)', 'Temperatura (°C)']

# Normalizar con Min-Max Scaling
for col in numeric_cols:
    min_val = np.min(df[col])
    max_val = np.max(df[col])
    df[col] = (df[col] - min_val) / (max_val - min_val)

print("DataFrame después de normalizar los datos:")
print(df.head())
```

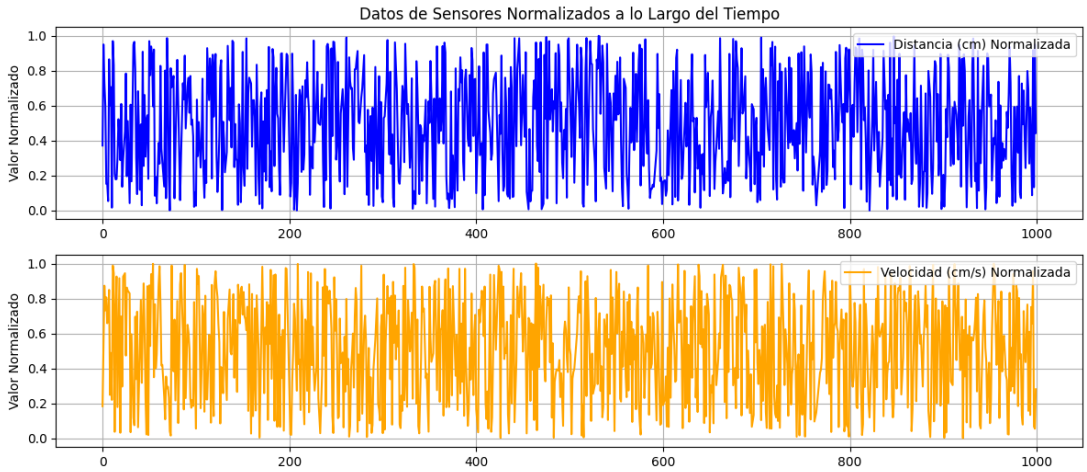
```
DataFrame después de normalizar los datos:
   Tiempo (s)  Distancia (cm)  Velocidad (cm/s)  Temperatura (°C)
0           0        0.371735          0.182609          0.262269
1           1        0.950755          0.540740          0.247509
2           2        0.730954          0.873049          0.908233
3           3        0.596960          0.731791          0.250082
4           4        0.152134          0.806411          0.272535
```

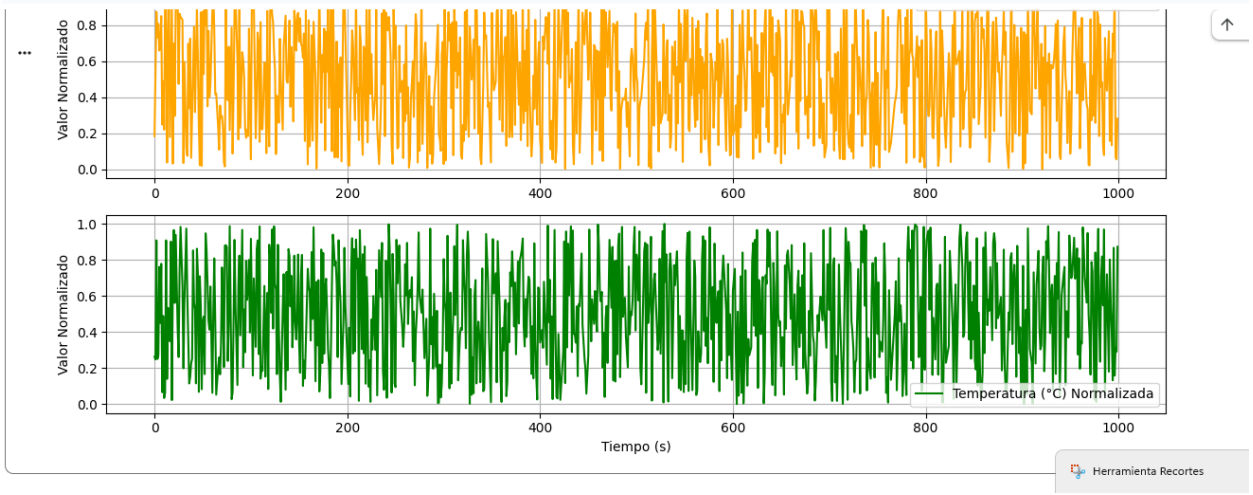
5. Visualización de los datos

Finalmente, se crearon tres gráficos alineados (subplots) para observar cómo evolucionaron la distancia, la velocidad y la temperatura a lo largo del tiempo. Al estar normalizados, es más sencillo detectar visualmente los momentos en que el robot alcanzó sus puntos máximos o mínimos de rendimiento.

```python
# Visualizar los datos normalizados
plt.figure(figsize=(12, 8))

plt.subplot(3, 1, 1)
plt.plot(df['Tiempo (s)'], df['Distancia (cm)'], label='Distancia (cm) Normalizada', color='blue')
plt.ylabel('Valor Normalizado')
plt.title('Datos de Sensores Normalizados a lo Largo del Tiempo')
plt.legend()
plt.grid(True)

plt.subplot(3, 1, 2)
plt.plot(df['Tiempo (s)'], df['Velocidad (cm/s)'], label='Velocidad (cm/s) Normalizada', color='orange')
plt.ylabel('Valor Normalizado')
plt.legend()
plt.grid(True)

plt.subplot(3, 1, 3)
plt.plot(df['Tiempo (s)'], df['Temperatura (°C)'], label='Temperatura (°C) Normalizada', color='green')
plt.xlabel('Tiempo (s)')
plt.ylabel('Valor Normalizado')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

```python
plt.grid(True)

plt.tight_layout()
plt.show()
```




Valor Normalizado
0.8
0.6
0.4
0.2
0.0
0
200
400
600
800
1000
Valor Normalizado
1.0
0.8
0.6
0.4
0.2
0.0
Temperatura (°C) Normalizada
Tiempo (s)
Herramienta Recortes

Preguntas

1. ¿Cómo afecta la limpieza de datos al análisis del dataset?

   La limpieza de datos elimina registros incompletos y corrige valores imposibles, lo que permite obtener resultados más confiables. Si no se realizara este paso, los errores de captura distorsionarían los cálculos y las gráficas, afectando la interpretación del comportamiento del robot.

2. ¿Por qué es importante la normalización en Machine Learning?

   La normalización es importante porque pone todas las variables en la misma escala. Esto evita que las variables con valores más grandes inffuyan de manera desproporcionada en los modelos y asegura un entrenamiento más estable y eficiente en algoritmos de Machine Learning.

3. ¿Cómo interpretarías los datos visualizados?

   Las gráficas muestran las variables transformadas en el rango [0,1], donde 0 representa el valor más bajo y 1 el más alto observado en la serie. Los picos indican momentos en que la variable alcanzó su valor máximo relativo, mientras que los valles representan los mínimos. Esto facilita comparar la evolución de la distancia, la velocidad y la temperatura a lo largo del tiempo.

GitHub

https://github.com/diegosaucedo2006-oss/INT.ART/blob/main/Actividades/A5_IA.ipynb

Conclusión grupal

La práctica permitió reforzar conocimientos sobre manejo de datos en Python y comprender la importancia de la limpieza y la normalización antes de aplicar cualquier análisis. Gracias a la visualización, se observaron patrones claros en las mediciones del robot, eliminando la influencia de datos defectuosos. En conjunto, este ejercicio nos enseñó que preparar los datos es un paso esencial en proyectos de inteligencia artificial y robótica, ya que garantiza la calidad y confiabilidad de los resultados obtenidos.

Referencias Bibliográficas

- Matthes, E. (2023). Python crash course. No starch press.
- https://www.w3schools.com/python/default.asp